

SUPPLY CHAIN SECURITY FRAMEWORK

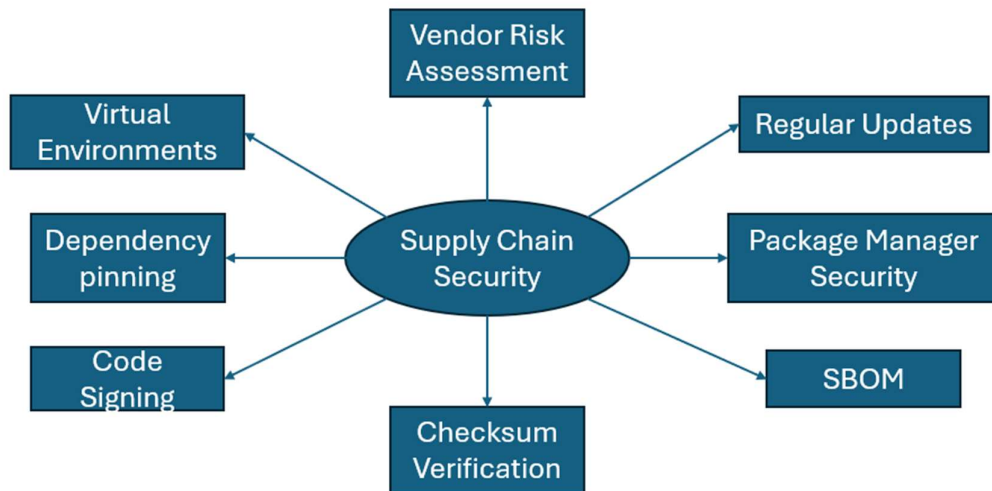
Contents

Introduction	2
Supply Chain Security Basic Protections	3
Supply Chain Security Advanced Protections.....	4
Supply Chain Security Framework Trade-offs	5

Introduction

This supply chain security framework is designed to guide businesses and rarely individuals in securing their digital supply chain. This framework was developed as most users need to download third-party software to perform certain activities. This means that attackers can use this supply chain to deliver malware onto a device. This framework's main purpose is to make it as costly as possible to compromise the supply chain. This supply chain security framework will cover basic and advanced protections while also providing the honest trade-offs that come with this level of security.

This supply chain security framework cannot promise perfect security or usability as no system can do so. However, this supply chain security framework can promise to try provide the foundation to make a robust and layered security system for digital supply chains. This framework is mainly targeted towards businesses but can be used by individuals with some adaptations. Make sure to adapt the protections based on your resources and threat model.



This spider diagram above shows some of the main protections involved in securing digital supply chains. For those who don't know, SBOM stands for software bill of materials. This means a complete inventory of components, dependencies and vulnerabilities. You can find more information about these protections in the sections below.

Supply Chain Security Basic Protections

- There are protections that each business or individual can implement for basic supply chain security.
- There are 5 basic protections that can be implemented.
- The first protection is dependency pinning.
- This is where you will lock dependencies to specific and verified versions that the vendor can use.
- This can be done by manually not updating or using a lock file via a command line using tools like npm or pip.
- Dependency pinning is important as it prevents unexpected updates that may introduce vulnerabilities and it also provides you with control over the update process.
- The second protection is checksum verification
- This is where you will verify the integrity of downloaded packages, software and files using verification hashes.
- This can be done by using commands using tools such as sha256sum, md5sum, CertUtil and Get-FileHash on a file or package
- After you have generated a hash, you must compare it to the hash provided by the vendor. If the file matches, its safe. If the file doesn't, assume its malicious.
- Checksum Verification is important to ensure that you don't download malicious software on your devices.
- The third protection is vendor risk assessment.
- This is where you will be evaluating and deciding whether to engage with vendors based on their security practices and breach history.
- This can be done by looking at their official website, looking at results of official independent audits and checking breach history via official sources.
- Vendor risk assessment is important as it allows you to avoid providers that are known to be insecure.
- The fourth protection is virtual environments.
- This is where you will be setting up isolated environments to run untrusted or new packages and dependencies in.
- This can be done by using virtual environments tools such as venv, virtualenv, conda, pipenv and poetry.
- Virtual environments are important as they ensure that any malicious packages or dependencies cannot affect the host system.
- The fifth protection is regular updates.
- This is where you will be updating these dependencies regularly to get the latest security patches.
- This can be done by using language-native tools such as pip, npm, yarn and poetry to ensure that packages and dependencies are updated.
- This is important to ensure that attackers cannot exploit outdated versions.

Supply Chain Security Advanced Protections

- There are also advanced protections that businesses can implement if they are facing more advanced threats.
- There are 5 advanced protections that can be implemented.
- The first protection is SBOM.
- SBOM stands for software bill of materials which is where a complete inventory of all components, dependencies and vulnerabilities is kept.
- This can be implemented by splitting this inventory into 3 sections and using structures to understand each item in the inventory.
- SBOM is important as it only for the rapid identification of vulnerabilities and ensures that there are no unauthorized dependencies being used.
- The second advanced protection is code signing.
- This is where all internal and external software's code must be cryptographically signed.
- This can be done by using signing tools such as OpenSSL, Microsoft SignTool, Jarsigner and GPG.
- Code signing is important as it ensures that both internal and external software are authentic as well as providing integrity monitoring.
- The third advanced protection is runtime integrity monitoring
- This is where you will be monitoring running applications for unauthorized changes in real-time.
- This can be done by using file integrity monitoring (FIM) , Process monitoring, Memory integrity, System call monitoring and Behavioural analysis.
- Runtime integrity monitoring is important because it can detect tempering in real-time allowing for quick incident responses.
- The fourth advanced protection is zero-trust for supply chain.
- This is where you assume every single third-party component cannot be trusted.
- This can be done by isolating supply chains on separate environments like VLAN's or local virtual environments, putting strict permissions and time-limiting their access.
- Zero-trust for supply chain is important as it ensures that supply chains are always checked and verified before, during and after usage.
- The fifth and final advanced protection is reproducible builds.
- This is where you build software in a form that produces the same output each time.
- This can be done by using tools such as docker, Nix and Bazel which provide containerised build environments.
- Approaches such as deterministic compilation, fixed dependencies, seeded randomness and building scripts can also be used.
- Reproducible builds are important as they prevent hidden modifications and allows for verification.

Supply Chain Security Framework Trade-offs

- This supply chain security framework has trade-offs just like any other system I have created.
- IT teams or individuals may face further implementation challenges as supply chain security is not easy to set up.
- I have identified 5 trade-offs.
- The first trade-off is time-consumption.
- This trade-off exists because it can take a large amount of time to set up both basic and advanced protections.
- This is because processes must be mirrored across multiple supply chains which can take long time especially in enterprise environments.
- This trade-off can be managed by working in teams if possible, using time management methods like time-blocking and securing only what's needed.
- The second trade-off is compatibility.
- This trade-off exists because not every system will be able to support such robust security features.
- This is because legacy systems weren't built with security in mind or aren't capable in general of handling the newest security features.
- This trade-off can be managed by testing security features on legacy systems, implementing what's possible and if possible, replace the legacy system.
- The third trade-off is complexity.
- This trade-off exists because some of the more advanced protections can be very difficult and confusing to set up correctly.
- This is because they require knowledge of certain tools and for you to navigate the different issues and your own unique environment.
- This trade-off can be managed by automating what can be automated, using structured learning processes and even hiring experts if needed.
- The fourth trade-off is cognitive load.
- This trade-off exists because verifying and authenticating each supply chain can be mentally exhausting.
- This means that IT teams or users may experience burnout if they are constantly paranoid and scanning for threats.
- This trade-off can be managed by automating certain security processes, having a rotation schedule for teams and using threat model for implementing features.
- The fifth trade-off is cost.
- This trade-off exists because some tools needed to set up advanced or basic protections can be expensive or costly to have.
- This means that some users or companies can not implement the security features they require.
- This trade-off can be managed via budget management and using free open-source tools where possible.